



PRINCESS NORA BINT ABDULRAHMAN UNIVERSITY

College of Computer and Information Sciences

Department of Information System

Design and Implementation of ETL and ELT Pipelines for YouTube Trending Data

Group 2

Section:63s

<i>Student Name</i>	<i>Student ID</i>
Jood Ahmed Bagazi	445008562
Mashael Mohammed Algazlan	445008598
Reem Turki Alotaibi	445008577
Shoug Alghaihab	445008607
Nourah Almuharb	445008623

Table of content

1. Chapter 1: Introduction

- 1.1 Background and motivation
- 1.2 Problem statement
- 1.3 Objectives of the Data engineering Project

2. Chapter 2: System Design and Architecture

- 2.1 Data sources and collection methods
- 2.2 Data Ingestion method
- 2.3 Data cleaning and transformation methods

3. Chapter 3: ETL Pipeline Design

- 3.1 Data pipeline architecture(ETL)
- 3.2 Extract: Collecting data from sources (APIs, databases, files)
- 3.3 Transform: Cleaning, filtering, aggregating data before loading.
- 3.4 Load: Storing transformed data into a data warehouse (e.g.,Sqlite)

4. Chapter 4: Implementation and Testing

- 4.1 Step-by-step ETL process execution using python program
- 4.2 Visualize using BI dashboard
- 4.3 Performance challenges and solutions

5. Chapter 5: (Phase 2) Introduction to ELT

- 5.1 Difference between ELT (Extract, Load, Transform) and ETL
- 5.2 Why ELT is used
- 5.3 Tools used as Datawarehouse and libraries for ELT

6. Chapter 6: ELT Pipeline Design

- 6.1 Data pipeline architecture (ELT)
- 6.2 Data sources: Collecting raw data
- 6.3 Load the raw data in datawarehouse
- 6.4 Transform: SQL based transformation

7. Chapter 7: Implementation & Results

- 7.1 Execution of ELT workflow
- 7.2 Visualize using BI dashboard
- 7.3 Automation of ELT pipeline using Task scheduler

7.4 Comparison of ETL vs. ELT (speed, scalability, efficiency)

7.5 Conclusion

7.6 References (IEEE format)

Table of figures

Figure 3.1: ETL Data Pipeline Architecture for YouTube Trending Videos Data

Figure 4.1.1: Extracting data from CSV and JSON

Figure 4.1.2: Extracting data from YouTube API

Figure 4.1.3: Data transformation and integration process

Figure 4.1.4: Loading data into SQLite database

Figure 4.1.5: Execution of the ETL pipeline

Figure 4.1.6: Output showing successful ETL pipeline execution, data transformation, and query results

Figure 4.1.7: Sample output from SQLite database showing stored video ..records after ETL processing

Figure 4.2: Power BI Dashboard for YouTube ETL Pipeline Analysis

Figure 7.1.1 Raw YouTube data extracted from CSV source

Figure 7.1.2 : Final YouTube data after transformation and loading into the PostgreSQL final table.

Figure 7.2.1: YouTube analytics dashboard created using Power BI.

Figure 7.3.1 : ELT pipeline scheduled using Windows Task Scheduler.

Figure 7.3.2: ELT script scheduled using Windows Task Scheduler.

List of table

Table 7.4 Comparison ETL vs ELT

1. Chapter 1: Introduction

1.1 Background and motivation

YouTube is one of the most widely used social media platforms worldwide, with billions of active users and a massive amount of content generated daily [1]. This large volume of data makes it an ideal source for data analysis and data engineering applications.

In data engineering, processing large-scale data efficiently is essential. Two common approaches are ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform), which differ in how and where data transformation is performed. These approaches impact system performance, scalability, and flexibility.

1.2 Problem statement

YouTube generates a large amount of daily data including views, likes, comments. The process of handling this data becomes difficult due to its .large size and continuous growth

The selection of an appropriate data processing method becomes crucial as traditional approaches may lead to slow performance, increased resource .consumption, and reduced system efficiency

Therefore, this project aims to implement both ETL and ELT pipelines on the same YouTube dataset and compare their performance in terms of execution speed, resource usage, scalability, and data quality. The goal is to identify the most efficient approach for large-scale data processing.

The project also includes data analysis to identify top video categories, .most viewed content, and user engagement

1.3 Objectives of the Data engineering Project

- To collect YouTube data from multiple sources, including datasets and APIs.
- To store the collected data in a structured format suitable for efficient processing and analysis.
- To design and implement an ETL pipeline to extract, transform, and load the data.
- To design and implement an ELT pipeline to extract, load, and then

Ensure data quality through validation and cleaning process.

- To compare ETL and ELT approaches in terms of execution performance, resource utilization, and scalability.
- To analyze the processed data to extract insights such as popular video categories, most viewed content, and user engagement.

2. Chapter 2: System Design and Architecture

2.1 Data sources and collection methods

The dataset used in this project was obtained from the Kaggle platform under the title YouTube Trending Data. The selected file for this project was USvideos.csv, which contains records of trending YouTube videos in the United States. The dataset includes valuable attributes such as video title, channel title, publish time, views, likes, dislikes, comment count, and category ID.

In addition to the CSV dataset, a JSON file named US_category_id.json was used to map category IDs into readable category names such as Music, Entertainment, Gaming, Education, and Sports. This improved the interpretability of the data during analysis.

Furthermore, the YouTube Data API was used as an additional external source to collect real-time metadata for selected trending videos. The API provided updated details such as current statistics, channel information, and publishing details.

This combination of static files (CSV and JSON) with dynamic API data allowed the project to simulate real-world multi-source data engineering environments.

2.2 Data Ingestion method

The project used a batch ingestion approach. In this method, data is collected and processed in scheduled batches instead of continuous real-time streaming.

For the ETL pipeline:

- Data was extracted from CSV and JSON files.
- Additional data was retrieved from YouTube API.
- The extracted data was processed in Python before loading into SQLite.

For the ELT pipeline:

- Raw data was first loaded directly into PostgreSQL staging tables.
- Transformations were executed later using SQL queries inside the database.

Batch ingestion was selected because it is suitable for structured datasets, easier to implement, and ideal for academic projects involving historical YouTube records.

2.3 Data cleaning and transformation methods

To ensure high data quality, several cleaning and transformation steps were applied:

Data Cleaning:

- Removed missing or null values using `dropna()`
- Removed duplicate records using `drop_duplicates()`
- Corrected inconsistent formatting in text columns
- Converted numerical columns into proper numeric types

Data Transformation:

- Converted `publish_time` into datetime format
- Mapped category IDs with category names using JSON data
- Selected only useful columns for analysis
- Merged data collected from API with local dataset
- Created a new calculated metric called `engagement_rate`

These transformations prepared the data for storage, querying, and visualization.

3. Chapter 3: ETL Pipeline Design

3.1 Data pipeline architecture(ETL)

The ETL data pipeline architecture starts with three data sources: USvideos.csv, US_category_id.json, and the YouTube Data API. The raw data is extracted using Python libraries such as pandas and json. The transform stage cleans the data by removing missing values and duplicates while mapping category IDs to category names and creating a new engagement column. The transformed data is loaded into the SQLite database youtube_etl.db while final_data.csv is exported for analysis and visualization purposes.

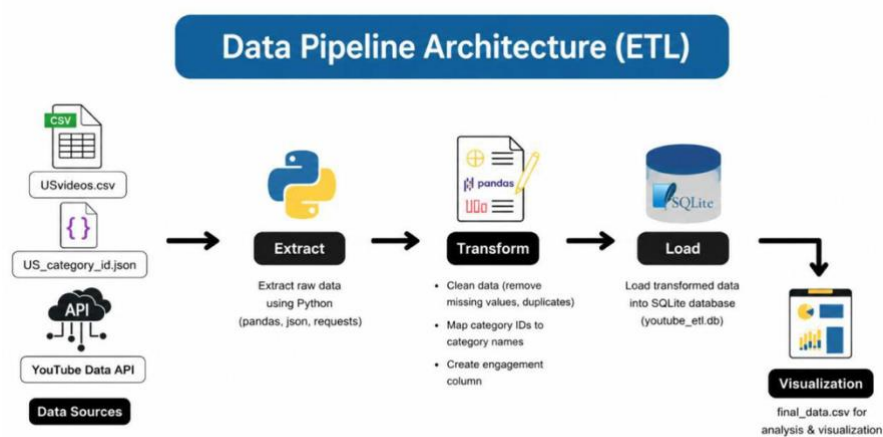


Figure 3.1: ETL Data Pipeline Architecture for YouTube Trending Videos Data

3.2 Extract: Collecting data from sources (APIs, databases, files)

The extraction phase integrates both local and external data sources to collect YouTube video data for further processing and analysis. This approach ensures that the dataset is both comprehensive and enriched with real-time information.

CSV File (USvideos.csv) This file contains the main dataset of trending YouTube videos. It includes key attributes such as *title*, *channel title*, *publish time*, *views*, *likes*, and *category_id*. This dataset serves as the primary source for analysis.

JSON File (US_category_id.json) This file provides a mapping between category IDs and their corresponding category names. It is used to convert numeric category identifiers into readable labels such as *Music*, *Entertainment*, and *Education*.

YouTube Data API The YouTube Data API was used to retrieve additional video data dynamically. Queries were executed for selected categories such as *Music*, *Gaming*, *Entertainment*, *Comedy*, *Science & Technology*, *People & Blogs*, and *Education*. The API provided real-time metadata including *video title*, *channel name*, and *publish time*, which enriched the overall dataset.

3.3 Transform: Cleaning, filtering, aggregating data before loading.

The transformation phase was applied to ensure consistency, accuracy, and usability of the dataset before loading it into storage.

Data Standardization:

- Selected relevant columns such as *title*, *channel title*, *publish time*, *views*, *likes*, and *category_name*.
- Converted the *publish_time* column into a datetime format for time-based analysis.

Category Mapping:

- Category IDs were mapped to their corresponding category names using the JSON file.

- This improved readability and enabled category-based analysis.

Data Cleaning:

- Missing values were removed using `dropna()` to ensure data quality.
- Duplicate records were removed using `drop_duplicates()` to eliminate redundancy.

Data Integration:

- The CSV dataset and API data were merged into a unified dataset using concatenation.
- This ensured a consistent structure across both data sources.

Feature Engineering:

- A new column called *engagement* was created using the formula:

$$engagement = \frac{likes}{views}$$

- This metric was used to measure user interaction with videos.

3.4 Load: Storing transformed data into a data warehouse (e.g.,Sqlite)

In the final stage of the ETL pipeline, the transformed dataset was stored in a SQLite database.

SQLite Database (youtube_etl.db):

The cleaned dataset was loaded into a table named *youtube_data*.

The `pandas.to_sql()` function was used to insert the data into the database.

Table Management:

The table was configured to overwrite existing data using the parameter `if_exists="replace"`, allowing iterative development and testing.

Storage Advantages:

SQLite was chosen due to its lightweight design, simplicity, and ease of integration with Python.

It provides efficient local storage suitable for small to medium-scale data engineering projects.

Exporting Data:

In addition to database storage, the final dataset was exported to a CSV file (*final_data.csv*).

This allows flexibility for validation, sharing, and visualization.

This loading phase ensured that the processed data was stored in a structured and accessible format, enabling efficient querying and analysis.

4. Chapter 4: Implementation and Testing

4.1 Step-by-step ETL process execution using python program

The pipeline steps:

4.1.1 Extract from CSV and JSON

```
1  import requests
2  import pandas as pd
3  import json
4  import sqlite3
5
6  CSV_FILE = "USvideos.csv"
7  JSON_FILE = "US_category_id.json"
8  DB_NAME = "youtube_etl.db"
9  TABLE_NAME = "youtube_data"
10 API_KEY = "AIzaSyBaDPjvs5Ia8FcFruDVCzz_jK1Jq-8Wf1E"
11
12 queries = {
13     "Music": "music",
14     "Gaming": "gaming",
15     "Entertainment": "entertainment",
16     "Comedy": "comedy",
17     "Science & Technology": "science technology",
18     "People & Blogs": "vlog",
19     "Education": "education"
20 }
21
22 def extract_csv():
23     df_csv = pd.read_csv(CSV_FILE)
24
25     with open(JSON_FILE) as f:
26         categories = json.load(f)
27
28     mapping = {}
29     for item in categories["items"]:
30         mapping[int(item["id"])] = item["snippet"]["title"]
31
32     df_csv["category_name"] = df_csv["category_id"].map(mapping)
33     print("CSV and JSON data extracted successfully")
34     return df_csv
```

Figure 4.1.1: Extracting data from CSV and JSON

4.1.2 Extract from YouTube API

```
37 def extract_api():
38     videos = []
39
40     for category_name, query in queries.items():
41         url = (
42             f"https://www.googleapis.com/youtube/v3/search?part=snippet&q={query}&type=video&maxResults=5&key={API_KEY}"
43         )
44
45         response = requests.get(url)
46         response.raise_for_status()
47         data = response.json()
48
49         if "items" not in data:
50             print(f"No API items returned for {category_name}")
51             continue
52
53         for item in data["items"]:
54             videos.append({
55                 "title": item["snippet"]["title"],
56                 "channel_title": item["snippet"]["channelTitle"],
57                 "publish_time": item["snippet"]["publishedAt"],
58                 "views": None,
59                 "likes": None,
60                 "category_name": category_name
61             })
62
63     df_api = pd.DataFrame(videos)
64
65     print("API data extracted successfully")
66     print("API rows:", len(df_api))
67     print(df_api.head())
68     return df_api
```

Figure 4.1.2: Extracting data from YouTube API

4.1.3 Data Transformation and Integration

```
64 def transform_data(df_csv, df_api):
65     df_csv = df_csv[[
66         "title",
67         "channel_title",
68         "publish_time",
69         "views",
70         "likes",
71         "category_name"
72     ]]
73
74     df_csv["publish_time"] = pd.to_datetime(df_csv["publish_time"], errors="coerce")
75     df_api["publish_time"] = pd.to_datetime(df_api["publish_time"], errors="coerce")
76
77     df_csv = df_csv.dropna()
78     df_csv = df_csv.drop_duplicates()
79
80     df_combined = pd.concat([df_csv, df_api], ignore_index=True)
81
82     df_combined["engagement"] = df_combined["likes"] / df_combined["views"]
83
84     print("Data cleaned, transformed, and merged successfully")
85     return df_combined
```

Figure 4.1.3: Data transformation and integration process

4.1.4 Load into SQLite Database

```
87 def load_to_sqlite(df):
88     conn = sqlite3.connect(DB_NAME)
89
90     df.to_sql(TABLE_NAME, conn, if_exists="replace", index=False)
91
92     print("Data loaded into SQLite successfully")
93
94     query = """
95     SELECT category_name, COUNT(*) AS total_videos
96     FROM youtube_data
97     GROUP BY category_name
98     ORDER BY total_videos DESC;
99     """
100
101     result = pd.read_sql(query, conn)
102     print(result)
103
104     conn.close()
```

Figure 4.1.4: Loading data into SQLite database

4.1.5 Saving Final Output

```
106 def main():
107     print("Starting ETL process...")
108
109     df_csv = extract_csv()
110     df_api = extract_api()
111
112     final_df = transform_data(df_csv, df_api)
113
114     final_df.to_csv("final_data.csv", index=False)
115     print("final_data.csv created successfully")
116
117     load_to_sqlite(final_df)
118
119     print("ETL process completed successfully")
120
121 if __name__ == "__main__":
122     main()
```

Figure 4.1.5: Execution of the ETL pipeline

4.1.6 VS code Output

This console output demonstrates the successful execution of the ETL pipeline.

The process begins with extracting data from multiple sources including CSV, JSON, and the YouTube API.

The extracted data is then cleaned, transformed, and merged into a unified dataset.

After transformation, the data is stored in an SQLite database, confirming the success of the loading phase.

Additionally, a sample SQL query is executed to validate the stored data by aggregating the number of videos per category.

The output confirms that all stages of the ETL pipeline were completed successfully and that the data is ready for further analysis and visualization.

```
(venv) PS C:\Users\user\Desktop\ML project> & "c:/Users/user/Desktop/ML project/venv/Scripts/python.exe" "c:/Users/user/Desktop/ML project/DE2.py"
Starting ETL process...
CSV and JSON data extracted successfully
API data extracted successfully
API rows: 35

```

	title	channel_title	publish_time	views	likes	category_name
0	Skusta Clee - Since Day One ft. Flow G (Lyrics...	Viral OPM	2026-05-04T14:45:48Z	None	None	Music
1	BEST RELAX TAGALOG LOVE STORY 2026 TAGALOG O...	Best Mix Reggae	2026-04-30T23:58:56Z	None	None	Music
2	2026 Trending Sinhala Band Nonstop Viral Hit...	Music Update	2026-05-04T10:15:02Z	None	None	Music
3	Douzi - Tal 3dabi قنداع لاط (Official Mus...	Douzi ذوزي	2026-05-02T21:26:08Z	None	None	Music
4	OPM Viral Songs 2026 🎧 Tagalog Love Songs With...	Serenata	2026-04-29T23:19:14Z	None	None	Music

```

Data cleaned, transformed, and merged successfully
final_data.csv created successfully
Data loaded into SQLite successfully

```

	category_name	total_videos
0	Entertainment	9949
1	Music	6472
2	Howto & Style	4142
3	Comedy	3458
4	People & Blogs	3213
5	News & Politics	2485
6	Science & Technology	2402
7	Film & Animation	2343
8	Sports	2172
9	Education	1660
10	Pets & Animals	920
11	Gaming	821
12	Travel & Events	401
13	Autos & Vehicles	384
14	Shows	57
15	Nonprofits & Activism	57

```

ETL process completed successfully
(venv) PS C:\Users\user\Desktop\ML project>

```

Figure 4.1.6: Output showing successful ETL pipeline execution, data transformation, and query results

4.1.7 Sqlite output

This sample output demonstrates the successful execution of the ETL pipeline using SQLite.

The dataset was extracted from CSV, JSON, and the YouTube API, then cleaned, transformed, and loaded into the SQLite database.

The output below shows a preview of the stored data using a SQL query. It includes key attributes such as video title, channel name, publish time, number of views, likes, category, and calculated engagement.

This confirms that the data has been correctly stored and is ready for further

```
C:\Users\user>cd c:/sqlite
c:\sqlite>sqlite3 "C:\Users\user\Desktop\ML project\youtube_etl.db"
SQLite version 3.53.0 2026-04-09 11:41:38
Enter ".help" for usage hints.
sqlite> .tables
youtube_data
sqlite> SELECT * FROM youtube_data LIMIT 5;
```

title	channel_title	publish_time	views	likes	category_name	engagement
WE WANT TO TALK ABOUT OUR MARRIAGE	CaseyNeistat	2017-11-13 17:13:01+00:00	748374	57527	People & Blogs	0.076869319351019677
The Trump Presidency: Last Week Tonight with John Oliver (HBO)	LastWeekTonight	2017-11-13 07:30:00+00:00	2418783	97185	Entertainment	0.040179296778586587
Racist Superman Rudy Mancuso, King Bach & Lele Pons	Rudy Mancuso	2017-11-12 19:05:24+00:00	3191434	146033	Comedy	0.045757800411977814
Nickelback Lyrics: Real or Fake?	Good Mythical Morning	2017-11-13 11:00:04+00:00	343168	10172	Entertainment	0.029641458411040655
I Dare You: GOING BALD!?	nigahiga	2017-11-12 18:01:41+00:00	2095731	132235	Entertainment	0.063097315447450078

```

sqlite>

```

Figure 4.1.7: Sample output from SQLite database showing stored video records after ETL processing

4.2 Visualize using BI dashboard

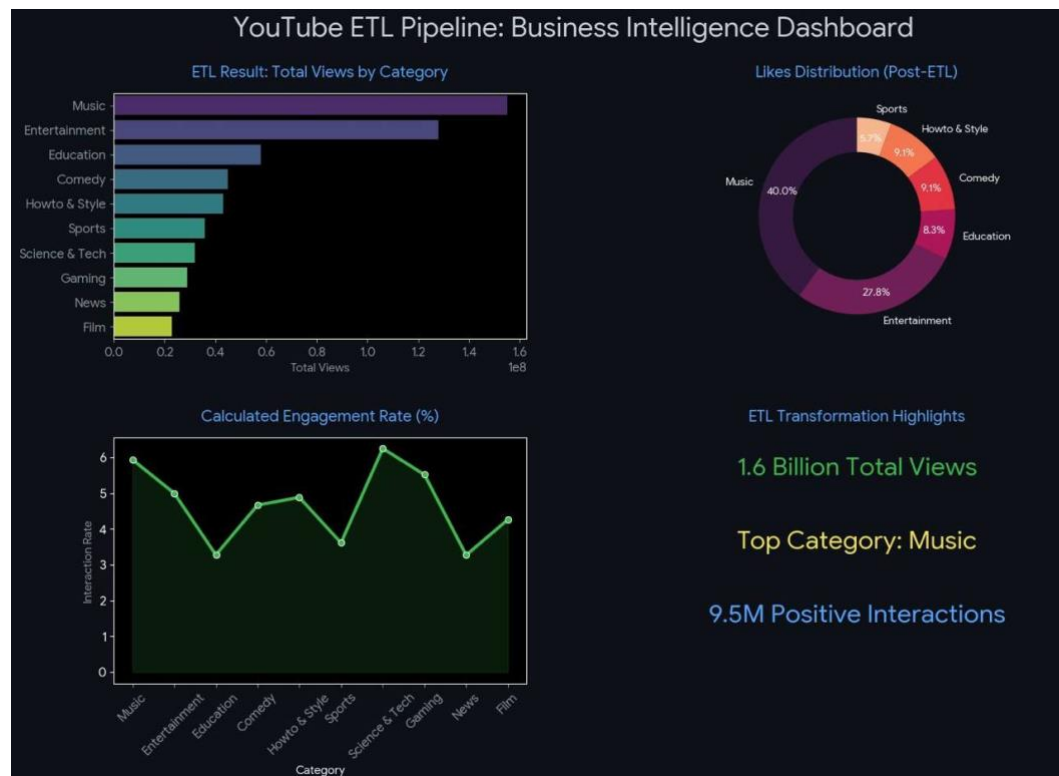


Figure 4.2: Power BI Dashboard for YouTube ETL Pipeline Analysis

4.3 Performance challenges and solutions

During the implementation of the ETL pipeline, several practical issues were encountered. These challenges were resolved through debugging and step-by-step corrections.

1. Incorrect SQLite Database Path

One of the main issues was that no tables appeared when running the `.tables` command in SQLite.

Cause:

The database was opened from a different directory (`C:\sqlite`), which caused SQLite to create a new empty database file.

Solution:

The issue was resolved by opening the correct database file using the full path:

```
sqlite3 "C:\Users\user\Desktop\ML project\youtube_etl.db"
```

This ensured access to the actual database containing the tables.

2. No Such Table Error

An error occurred:

no such table: youtube_data

Cause:

The database file being accessed did not contain the table, either because it was empty or not properly loaded.

Solution:

Running the ETL script again ensured that the table was created using:

```
df.to_sql("youtube_data", conn, if_exists="replace", index=False)
```

After that, the table appeared correctly.

3. API Key Syntax Errors

Errors such as:

- NameError
- SyntaxError

Cause:

The API key was not written correctly inside the URL string (missing quotes or broken f-string).

Solution:

The API key was correctly placed inside quotes:

```
key = "YOUR_API_KEY"
```

```
url = f"...&key={key}"
```

4. Large API Output in Console

The console displayed a very large JSON response from the API, making it difficult to read and analyze the results.

Solution:

The unnecessary `print(data)` statement was removed, and only important outputs such as the number of rows and a preview of the dataset (`df.head()`) were displayed.

5. Chapter 5: (Phase 2) Introduction to ELT

5.1 Difference between ELT (Extract, Load, Transform) and ETL

ETL and ELT are two widely used approaches in data engineering for processing and storing data [2], [3]. The main difference between them lies in the order of the transformation step.

In the ETL approach, data is first extracted from different sources such as CSV files, JSON files, and APIs. After that, the data is cleaned and transformed before being loaded into the data warehouse. In this project, the YouTube dataset was processed using ETL by removing missing values, handling duplicates, mapping category IDs to names, and calculating engagement before storing the final dataset in SQLite.

On the other hand, the ELT approach follows a different sequence. Data is extracted and loaded directly into the database in its raw form without applying transformations. After loading, SQL queries are used to clean and transform the data inside the database. This allows the raw data to be preserved and reused when needed.

This difference shows that ETL focuses on preparing data before storage, while ELT provides more flexibility by allowing transformations after loading, which is useful for handling large and dynamic datasets [2].

5.2 Why ELT is used

ELT is widely used in modern data systems because it allows storing raw data before applying any transformations [2]. This approach provides flexibility, as transformations can be performed later depending on the analysis requirements [3].

In this project, ELT is useful because YouTube data comes from multiple sources and may change over time. By loading the raw data first, it becomes easier to modify the transformation logic later without needing to repeat the extraction process. For example, additional calculations such as engagement metrics or filtering by category can be applied using SQL queries after loading the data.

Another reason for using ELT is to support the comparison between ETL and ELT within the same project. By implementing both approaches on the same dataset, it becomes possible to evaluate their performance in terms of speed, scalability, and efficiency. This comparison helps in understanding when each approach should be used in real-world applications [2], [3].

5.3 Tools used as Datawarehouse and libraries for ELT

The ELT pipeline in this project was implemented using a set of simple and practical tools.

SQLite was used as the data warehouse to store the raw YouTube data. It is a lightweight and easy-to-use database, making it suitable for small to medium-scale projects and academic applications [5].

Python was used to handle the extraction and loading process. It was responsible for collecting data from CSV files, JSON files, and the YouTube API, then inserting the raw data into the database.

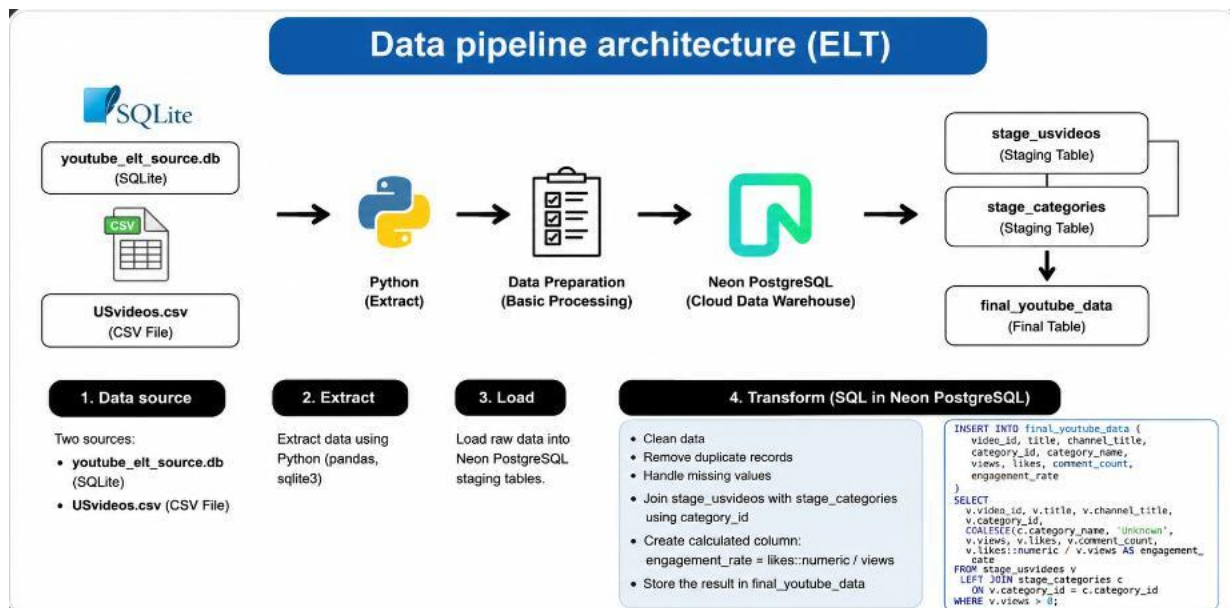
Pandas was used to read and organize the data before loading. Although ELT focuses on transformation inside the database, pandas plays an important role in preparing the data during the extraction phase [6].

The **sqlite3** library was used to connect Python with the SQLite database and execute SQL commands.

Finally, **SQL** was used to perform transformations after loading the raw data. This includes filtering records, selecting relevant columns, and preparing the dataset for analysis and visualization. Using SQL for transformation makes the ELT process more flexible and efficient when dealing with changing data requirements [2].

6.Chapter 6: ELT Pipeline Design

6.1 Data pipeline architecture (ELT)



The ELT data pipeline architecture begins with two data sources: the youtube_elt_source.db SQLite database and the USvideos.csv file. The SQLite database stores category information such as category_id, category_name, and category_type, while the CSV file contains YouTube trending video data including video title, channel title, views, likes, comments, and category IDs. Using Python libraries such as pandas and sqlite3, the raw data is extracted from both sources and prepared for loading into Neon PostgreSQL, a cloud-based PostgreSQL data warehouse. The extracted datasets are loaded into staging tables named stage_usvideos and stage_categories. After the loading process, SQL transformations are performed directly inside Neon PostgreSQL to clean the data, remove duplicate records, handle missing values, and join both staging tables using category_id. In addition, a calculated column called engagement_rate is generated by dividing the number of likes by the number of views to measure audience interaction with videos. Finally, the transformed and organized data is stored in the final_youtube_data table, making it ready for analytics, reporting, and visualization.

6.2 Data sources: Collecting raw data

Extract^[1]_{SEP} The initial stage of the ELT pipeline involves extracting raw data from two YouTube data sources which include USvideos.csv and youtube_elt_source.db.^[1]_{SEP}
The file USvideos.csv holds essential information about trending videos which includes the video title ,channel title ,publish time ,view count ,like count ,comment count and category_id.

The SQLite database youtube_elt_source.db holds category information created manually for the ELT pipeline. It includes category_id, category_name, and category_type. This source is used to map each video category_id from the USvideos.csv file to a readable category name such as Music, Gaming, Entertainment, and Education.

At this stage, the data is extracted in raw form without major cleaning or transformation processes

6.2.1 Extracting raw data

```
import pandas as pd
import sqlite3
from sqlalchemy import create_engine

sqlite_path = "/Users/mashaalqzlan/Desktop/sqlite-tools-osx-arm64-3510200/youtube_elt_source.db"
csv_path = "USvideos.csv"
conn_sqlite = sqlite3.connect(sqlite_path)
df_categories = pd.read_sql_query("SELECT * FROM legacy_categories", conn_sqlite)
conn_sqlite.close()

df_videos = pd.read_csv(csv_path)
```

6.3 Load the raw data in datawarehouse

The **Load** phase is the stage where the extracted raw data is loaded into the cloud database (Neon PostgreSQL) before transformation begins. According to the file, the project loads two datasets into Neon staging tables:

- stage_usvideos which contains YouTube trending video information from USvideos.csv
- stage_categories which contains category information from the SQLite database.

The purpose of the staging tables is:

- Store raw extracted data
- Prepare data for SQL transformations
- Separate raw data from cleaned analytical data

Not to mention that we used Pandas and SQLAlchemy to push the datasets into Neon PostgreSQL

6.3.1 Load the raw data in datawarehouse

```
df_videos = pd.read_csv(csv_path)

engine = create_engine(
    "postgresql+psycopg2://neondb_owner:npg_TeCRUh4PYp1L@ep-flat-butterfly-an76x6b1-pooler.c-6.us-east-1.aws.neon.tech/neondb?"
)
df_categories.to_sql("stage_categories", engine, if_exists="replace", index=False)
df_videos.to_sql("stage_usvideos", engine, if_exists="replace", index=False)

print("Data loaded into Neon successfully")
```

6.4 Transform: SQL based transformation

In the final stage of the ELT pipeline, the raw data stored in Neon PostgreSQL was transformed using SQL queries.

The data transformation process included cleaning the data, removing duplicate records, handling missing values, and merging the video data with category data using `category_id`. The new calculated column engagement was created by dividing the total likes by the total views to measure user interaction with videos.

The raw data became clean and organized for analysis and visualization after the transformation process.

6.4.1 Transform: SQL based transformation Neon code

```
SELECT * FROM stage_categories LIMIT 10;
SELECT * FROM stage_usvideos LIMIT 10;

DROP TABLE IF EXISTS final_youtube_data;

CREATE TABLE final_youtube_data (
  video_id TEXT,
  title TEXT,
  channel_title TEXT,
  category_id BIGINT,
  category_name TEXT,
  views BIGINT,
  likes BIGINT,
  comment_count BIGINT,
  engagement_rate NUMERIC
);

INSERT INTO final_youtube_data (
  video_id,
  title,
  channel_title,
  category_id,
  category_name,
  views,
  likes,
  comment_count,
  engagement_rate
)
SELECT
  v.video_id,
  v.title,
  v.channel_title,
  v.category_id,
  COALESCE(c.category_name, 'Unknown') AS category_name,
  v.views,
  v.likes,
  v.comment_count,
  v.likes::numeric / v.views AS engagement_rate
FROM stage_usvideos v
LEFT JOIN stage_categories c
ON v.category_id = c.category_id
WHERE v.views > 0;

SELECT * FROM final_youtube_data LIMIT 10;
```


6.4.2 Transform: SQL based transformation “Output”

1: SELECT 10 2: SELECT 10 3: DROP 4: CREATE 5: INSERT 40949 6: SELECT 10

#	video_id	trending_date	title	channel_title	category_id	publish_time	tags
			MARRIAGE				
2	1ZAPwfrtAFY	17.14.11	The Trump Presidency: Last Week Tonight with John Oliver (HBO)	LastWeekTonight	24	2017-11-13T07:30:00.000Z	last week tonight trump presidency last week toni
3	5qpjK5DgCt4	17.14.11	Racist Superman Rudy Mancuso, King Bach & Lele Pons	Rudy Mancuso	23	2017-11-12T19:05:24.000Z	racist superman "rudy" mancuso "king" bach "ra video" iphone x by pineapple "lelepons" hannahstocking "rudymancuso My Driver's License Lele Pons"
4	puqawrEC7tY	17.14.11	Nickelback Lyrics: Real or Fake?	Good Mythical Morning	24	2017-11-13T11:00:04.000Z	rhett and link gmm "good mythical morning" rhett morning "Season 12" nickelback lyrics "nickelba nickelback" gmm nickelback "lyrics (website catej kroeger) canada "music (industry)" mythical "gr
5	d380meD0W0M	17.14.11	I Dare You: GOING BALD!?	nigahiga	24	2017-11-12T18:01:41.000Z	ryan higa higatv "nigahiga" i dare you "idy"
6	kga045Sya04	17.14.11	The New SpotMini	BostonDynamics	28	2017-11-13T20:09:58.000Z	Robots "Boston Dynamics" SpotMini "Legged L
7	n30K5CwLhS4	17.14.11	Nick Andopolis: Drummer	FaeryInLoveInc	1	2011-05-29T17:03:12.000Z	freaks and geeks jason segel "judd apatow" um

1: SELECT 10 2: SELECT 10 3: DROP 4: CREATE 5: INSERT 40949 6: SELECT 10

#	category_id	category_name	category_type
1	1	Film & Animation	Entertainment
2	2	Autos & Vehicles	Lifestyle
3	10	Music	Entertainment
4	15	Pets & Animals	Lifestyle
5	17	Sports	Entertainment
6	19	Travel & Events	Lifestyle
7	20	Gaming	Entertainment
8	22	People & Blogs	Lifestyle
9	23	Comedy	Entertainment
10	24	Entertainment	Entertainment

1: SELECT 102: SELECT 103: DROP4: CREATE5: INSERT 409496: SELECT 10

#	video_id	title	channel_title	category_id	category_name	views	likes	comment_count	engagement_rate
1	2kyS6SvSYSE	WE WANT TO TALK ABOUT OUR MARRIAGE	CaseyNeistat	22	People & Blogs	748374	57527	15954	0.076869319351019
2	1ZAPwfrtAFY	The Trump Presidency: Last Week Tonight with John Oliver (HBO)	LastWeekTonight	24	Entertainment	2418783	97185	12703	0.040179296778586
3	5qpjK5DgCt4	Racist Superman Rudy Mancuso, King Bach & Lele Pons	Rudy Mancuso	23	Comedy	3191434	146033	8181	0.045757800411977
4	puqaWrEC7tY	Nickelback Lyrics: Real or Fake?	Good Mythical Morning	24	Entertainment	343168	10172	2146	0.029641458411040
5	d380meD0w0M	I Dare You: GOING BALD!?	nigahiga	24	Entertainment	2095731	132235	17518	0.063097315447450
6	kga045Sya04	The New SpotMini	BostonDynamics	28	Science & Technology	75752	9419	1230	0.124339951420424
7	n38k5CwLhS4	Nick Andopolis: Drummer	FaeryInLoveInc	1	Film & Animation	50867	715	246	0.014056264375724
8	Jidk006uu-0	Granulated Sugar From Honey	Cody's Lab Backup	22	People & Blogs	71415	4483	420	0.0627739271138

mashaelgz1@gmail.com

Free

ELT

All OK

Ask AI

Upgrade

M

PROJECT

Dashboard

Branches

Integrations

Settings

BRANCH

production

Overview

Monitoring

SQL Editor

Tables

Backup & Restore

Data Masking

BETA

APP BACKEND

Data API

Collapse menu

SQL Editor

production

Saved

History

May 8, 2026 - 12:02am

create YouTube data table with engagement metrics

May 8, 2026 - 12:00am

preview stage categories and videos tables

May 7, 2026 - 11:52pm

first 10 stage categories

May 7, 2026 - 11:50pm

compare final and stage customers

Apr 23, 2026 - 1:14pm

compare final and stage customers

Apr 23, 2026 - 1:13pm

create final_customers table wi

Untitled

Save

Primary

Active

neondb

Connected (6 queries)

Run

Explain

Analyze

194ms10 rows

1: SELECT 102: SELECT 103: DROP4: CREATE5: INSERT 409496: SELECT 10

#	video_id	title	channel_title	category_id
1	2kyS6SvSYSE	WE WANT TO TALK ABOUT OUR MARRIAGE	CaseyNeistat	22
2	1ZAPwfrtAFY	The Trump Presidency: Last Week Tonight with John Oliver	LastWeekTonight	24

1

7. Chapter 7: Implementation & Results

7.1 Execution of ELT workflow

The ELT pipeline was executed successfully following the workflow design and architecture presented in Chapter 6. The implementation process included four main stages: extraction, validation, loading, and transformation. The workflow was implemented using Python, PostgreSQL (Neon), SQLAlchemy, and SQL queries executed through the Neon SQL Editor.

Tools and Technologies Used:

- Python (Visual Studio Code): Used to implement and execute the ELT pipeline script.
- Pandas: Used for reading and processing the CSV dataset.
- SQLAlchemy: Used for connecting Python with the Neon PostgreSQL database.
- Neon PostgreSQL: Used for storing staging and final transformed tables.
- SQL Queries: Used to clean, transform, and structure the data inside the database.
- Windows Task Scheduler: Used to automate the ELT workflow execution.

7.1.1 Extract

The extraction stage focused on collecting raw YouTube trending video data from the CSV dataset. The dataset was loaded into Python using the Pandas library through the `pd.read_csv()` function. The extracted data included several attributes such as video title, channel title, category ID, views, likes, comments, and publish date. The raw dataset was reviewed before applying any transformations to ensure that the data was loaded correctly and prepared for the next ELT stages.

7.1.2 Validate

Before loading the data into the PostgreSQL database, basic validation checks were performed to ensure data quality and consistency. The validation process focused on checking missing values and verifying that important columns contained valid and complete information.

The validation stage helped identify potential data quality issues before inserting the dataset into the staging tables. The validation results confirmed that the dataset was processed successfully without major errors.

7.1.3 Load to PostgreSQL / Neon

After completing the validation process, the dataset was loaded into the Neon PostgreSQL database using SQLAlchemy and the `to_sql()` function in Python. The data was inserted into staging tables including `stage_usvideos` and `stage_categories`. This stage confirmed that the connection between Python and the Neon PostgreSQL database was successful and that the records were inserted correctly into the staging tables.

7.1.4 Transform (SQL Queries)

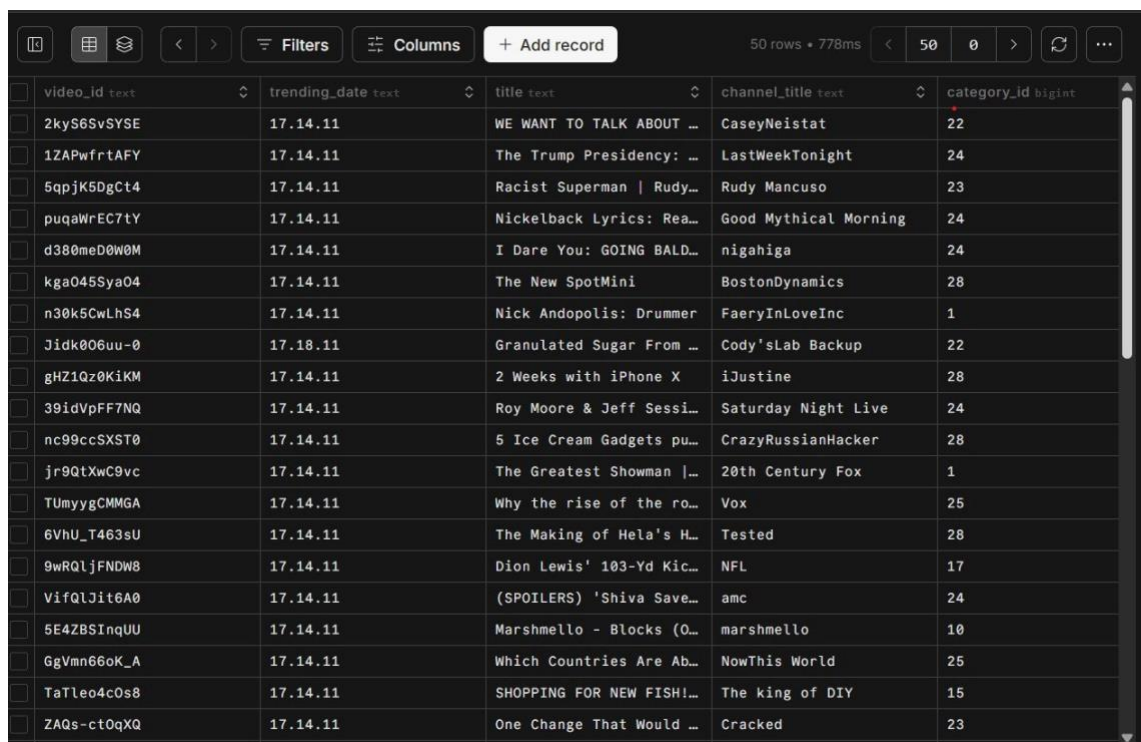
In the transformation stage, SQL queries were executed on the staging tables to clean and organize the data. Several transformation operations were applied, including standardizing category information, organizing text fields, and preparing the dataset for analysis and visualization.

The transformation process also improved the structure of the dataset and prepared the final tables for dashboard creation and analytical reporting.

7.1.5 Final Transformed Data

After completing the transformation process, the cleaned and structured dataset was stored in the final PostgreSQL tables. The final transformed data was then used for visualization and dashboard development in Power BI.

The final tables provided organized YouTube trending video data that supported analytical insights related to views, likes, comments, video categories, and channel performance.



video_id text	trending_date text	title text	channel_title text	category_id bigint
2kyS6SvSYSE	17.14.11	WE WANT TO TALK ABOUT ...	CaseyNeistat	22
1ZAPwfrtAFY	17.14.11	The Trump Presidency: ...	LastWeekTonight	24
5qpjK5DgCt4	17.14.11	Racist Superman Rudy...	Rudy Mancuso	23
puqaWrEC7tY	17.14.11	Nickelback Lyrics: Rea...	Good Mythical Morning	24
d380meD0W0M	17.14.11	I Dare You: GOING BALD...	nigahiga	24
kga045Sya04	17.14.11	The New SpotMini	BostonDynamics	28
n30k5CwLhS4	17.14.11	Nick Andopolis: Drummer	FaeryInLoveInc	1
Jidk006uu-0	17.18.11	Granulated Sugar From ...	Cody'sLab Backup	22
gHZ1Qz0KiKM	17.14.11	2 Weeks with iPhone X	iJustine	28
39idVpFF7NQ	17.14.11	Roy Moore & Jeff Sessi...	Saturday Night Live	24
nc99ccSXST0	17.14.11	5 Ice Cream Gadgets pu...	CrazyRussianHacker	28
j9r9QtXwC9vc	17.14.11	The Greatest Showman ...	20th Century Fox	1
TUmyygCMMGA	17.14.11	Why the rise of the ro...	Vox	25
6VhU_T463sU	17.14.11	The Making of Hela's H...	Tested	28
9wRQljFNDW8	17.14.11	Dion Lewis' 103-Yd Kic...	NFL	17
VifQLJit6A0	17.14.11	(SPOILERS) 'Shiva Save...	amc	24
5E4ZBSInqUU	17.14.11	Marshmello - Blocks (O...	marshmello	10
GgVmn66oK_A	17.14.11	Which Countries Are Ab...	NowThis World	25
TaTleo4c0s8	17.14.11	SHOPPING FOR NEW FISH!	The king of DIY	15
ZAQs-ct0qXQ	17.14.11	One Change That Would ...	Cracked	23

Figure 7.1.1 Raw YouTube data extracted from CSV source

```

36     v.likes,
37     v.comment_count,
38     v.likes::numeric / v.views AS engagement_rate
39 FROM stage_usvideos v
40 LEFT JOIN stage_categories c
41 ON v.category_id = c.category_id
42 WHERE v.views > 0;
43
44 SELECT * FROM final_youtube_data LIMIT 10;

```

Connected (6 queries)

Run Explain Analyze 247ms 10 rows

1: SELECT 10 2: SELECT 10 3: DROP 4: CREATE 5: INSERT 40949 6: SELECT 10

#	category_id	category_name	category_type
1	1	Film & Animation	Entertainment
2	2	Autos & Vehicles	Lifestyle
3	10	Music	Entertainment
4	15	Pets & Animals	Lifestyle
5	17	Sports	Entertainment
6	19	Travel & Events	Lifestyle
7	20	Gaming	Entertainment
8	22	People & Blogs	Lifestyle
9	23	Comedy	Entertainment
10	24	Entertainment	Entertainment

Figure 7.1.2 : Final YouTube data after transformation and loading into the PostgreSQL final table.

7.2 Visualize using BI dashboard

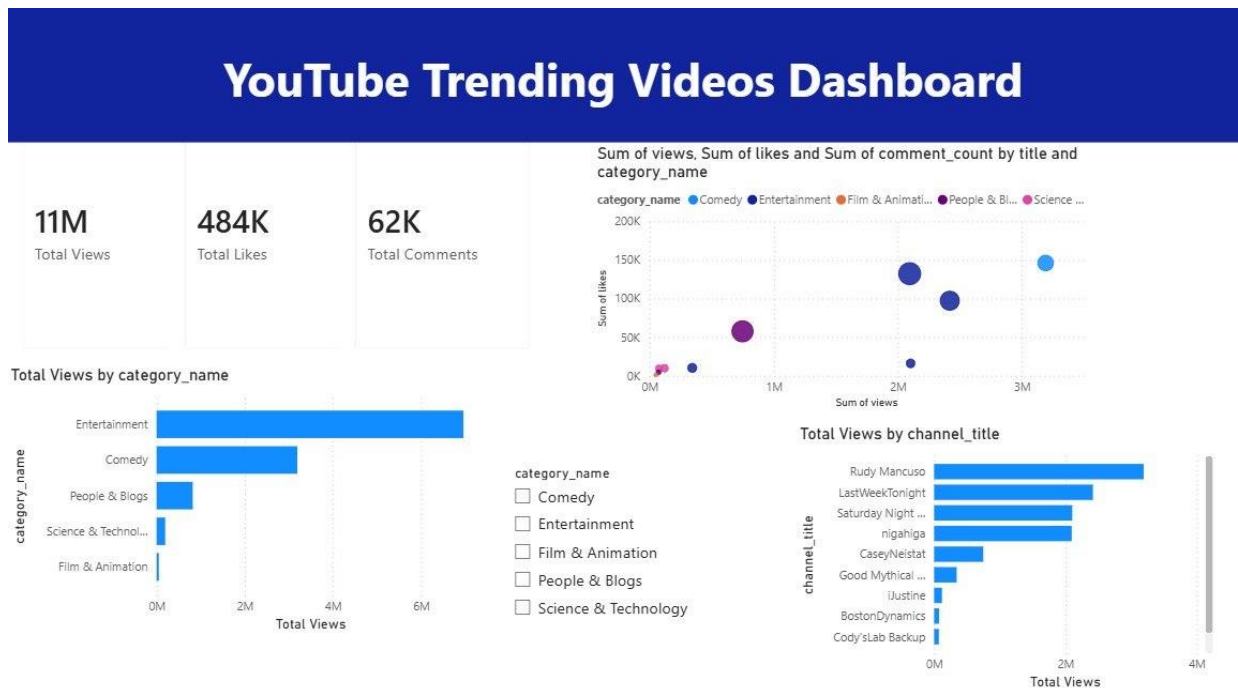


Figure 7.2.1: YouTube analytics dashboard created using Power BI.

7.3 Automation of ELT pipeline using Task scheduler

To automate the ELT pipeline and avoid running the process manually each time, Windows Task Scheduler was used. This automation allows the ELT workflow to run automatically at a scheduled time.

First, the Python ELT script was tested in Visual Studio Code to ensure that the extraction, loading, and transformation processes worked correctly and that the data was successfully loaded into the Neon PostgreSQL database. After confirming successful execution, Task Scheduler was opened and a basic task was created to run the script automatically every day.

The Python interpreter path was selected, and the ELT script path was added in the task configuration settings. After completing the setup, the task was tested manually from Task Scheduler to verify successful execution. The output messages confirmed that the script executed correctly and that the data pipeline completed successfully without runtime errors.

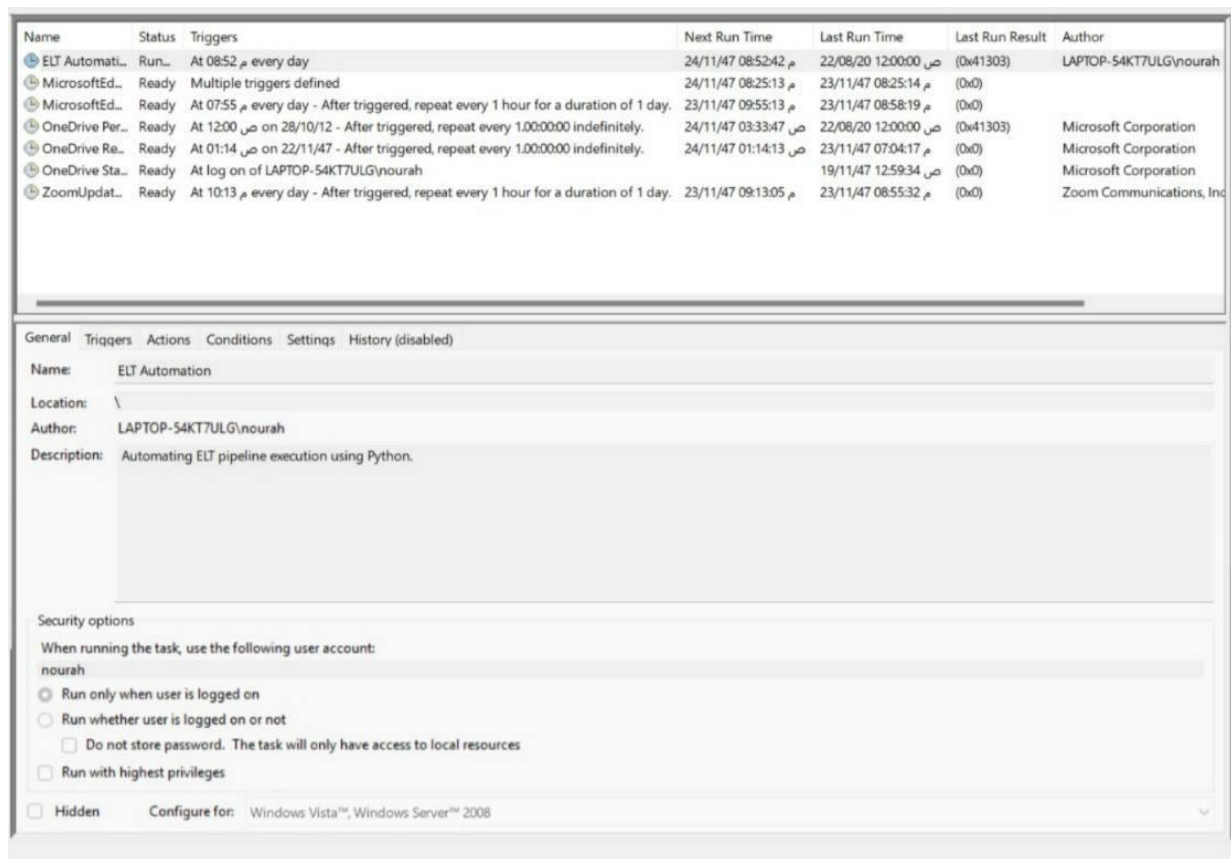


Figure 7.3.1 : ELT pipeline scheduled using Windows Task Scheduler

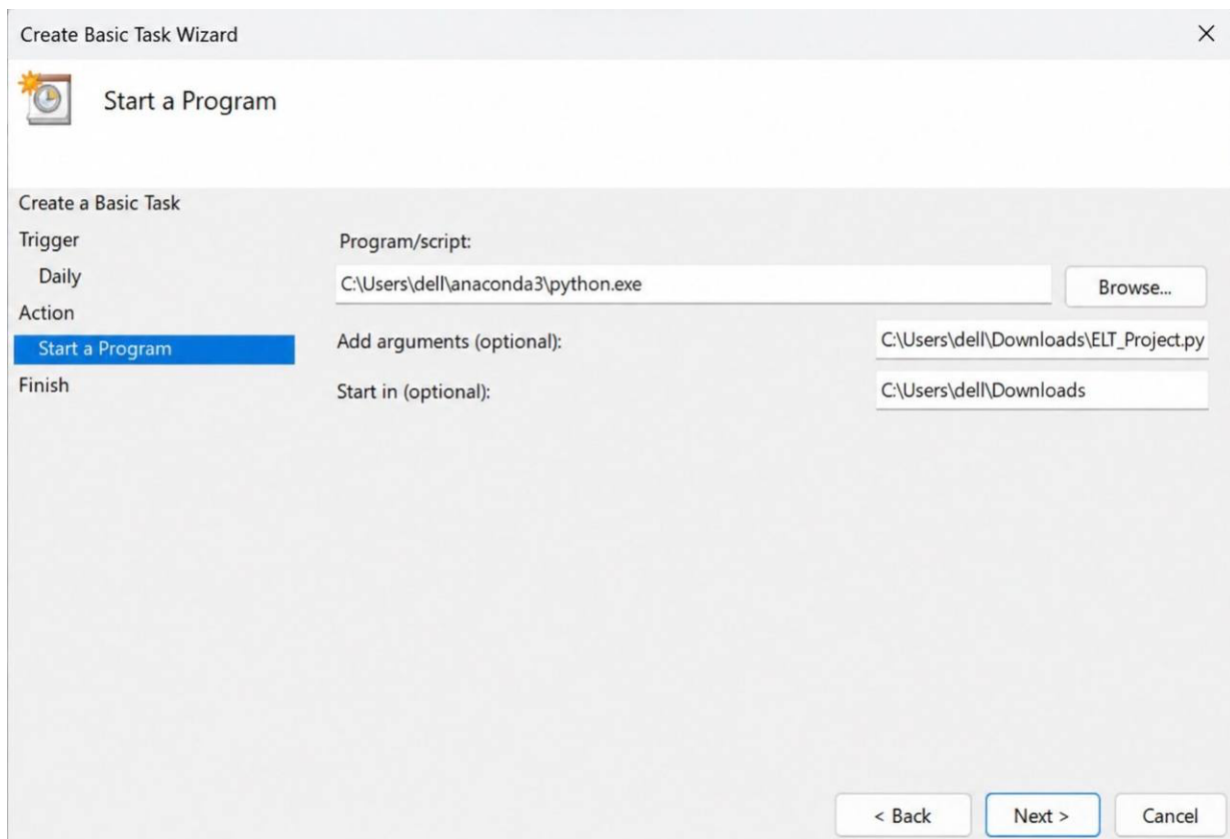


Figure 7.3.2: ELT script scheduled using Windows Task Scheduler.

7.4 Comparison of ETL vs. ELT (speed, scalability, efficiency)

Performance Comparison	ETL	ELT
Speed	6 Sec	0.5 Sec
Scalability	Less scalable	More scalable
Efficiency	Lower	Higher

7.4.1 Speed

ETL: ETL is slower because transformations are performed before loading the data into the database, which increases processing time.

ELT: ELT is faster because the data is loaded into PostgreSQL first, and transformations are executed directly inside the database using SQL queries.

7.4.2 Scalability

ETL: ETL is less scalable because it depends on external processing before loading the data.

ELT: ELT is more scalable because PostgreSQL and cloud databases can process large datasets more efficiently.

7.4.3 Efficiency

ETL: ETL requires additional processing steps and more data movement between systems.

ELT: ELT is more efficient because transformations are performed directly inside the database with fewer processing steps.

7.5 Conclusion

This project presented the design and implementation of both ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) pipelines using YouTube trending video data collected from CSV files, JSON files, and the YouTube Data API. The project demonstrated how modern data engineering techniques can be used to process, organize, and analyze large-scale social media data efficiently.

The ETL pipeline focused on transforming and cleaning the data before loading it into the SQLite database using Python and Pandas. This approach provided greater control over preprocessing and data quality management before storage. However, the transformation process required additional processing time and depended heavily on external tools and memory resources.

In contrast, the ELT pipeline loaded the raw data directly into the Neon PostgreSQL database before applying SQL-based transformations. This approach simplified the workflow and improved scalability by allowing transformations to be executed inside the database environment. SQL queries were used to clean the data, organize category information, and calculate engagement metrics efficiently.

The project also included data visualization using Power BI dashboards to provide analytical insights related to video views, likes, comments, categories, and channel performance. In addition, Windows Task Scheduler was used to automate the ELT workflow execution, making the pipeline more practical and efficient for repeated processing tasks.

Overall, the comparison showed that ETL is suitable for smaller datasets and controlled preprocessing tasks, while ELT is more flexible and scalable for modern cloud-based data systems. This project provided practical experience in building real-world data engineering pipelines and highlighted the importance of selecting the appropriate architecture depending on data size, processing requirements, and system scalability.

7.6 References (IEEE format)

- [1] Statista, “Most Used Social Networks 2025, by Number of Users,” Apr. 08, 2026. [Online]. Available: <https://www.statista.com/statistics/272014/global-social-networks-ranked-by-number-of-users>
- [2] Microsoft, “ETL vs ELT: What's the Difference?,” Microsoft Learn, 2023. [Online]. Available: <https://learn.microsoft.com/en-us/azure/architecture/data-guide/relational-data/etl>. [Accessed: May 5, 2026].
- [3] AWS, “What is ETL (Extract, Transform, Load)?,” Amazon Web Services, 2023. [Online]. Available: <https://aws.amazon.com/what-is/etl/>. [Accessed: May 5, 2026].
- [4] Google, “YouTube Data API Overview,” Google Developers, 2024. [Online]. Available: <https://developers.google.com/youtube/v3>. [Accessed: May 5, 2026].
- [5] SQLite, “About SQLite,” SQLite Official Documentation, 2024. [Online]. Available: <https://www.sqlite.org/about.html>. [Accessed: May 5, 2026].
- [6] W. McKinney, Python for Data Analysis, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2017.